Manasha Sekar

Step 2: DynamoDB Implementation

Part 1: DynamoDB Table Design

NoSQL Access Pattern Analysis

Shopping carts are a natural fit for DynamoDB because all three API operations share a single access pattern: retrieve by cart ID. There are no complex joins, no multi-table queries, and no ad-hoc filters exactly the kind of workload DynamoDB is optimized for.

Design Decisions

Partition Key Strategy cart_id (UUID string) was chosen as the partition key. UUIDs are randomly generated, which ensures requests are distributed evenly across DynamoDB partitions with no hot spots. Sequential integer IDs (like MySQL auto-increment) were considered but rejected because sequential keys concentrate writes on the most recently created partition.

Sort Key No sort key was needed. All three access patterns only ever retrieve a single cart and there is no range query or multi-item retrieval within a partition.

Table Structure A single table shopping_carts_dynamo was used. MySQL required four tables (shopping_carts, shopping_cart_items, products, orders) due to normalization rules. DynamoDB has no joins, so all cart data including items is stored in one record.

Attribute Design — Items Embedded Cart items are stored as a list of maps inside the cart record:

json
```
{
  "cart_id": "ecbbf081-869b-4945-a07f-18854bda886d",
  "customer_id": 1,
  "status": "active",
  "items": [
    { "product_id": 12345, "quantity": 2 }
  ],
  "created_at": "2026-03-22T23:36:41Z",
  "updated_at": "2026-03-22T23:36:43Z"
}
```

This means a single GetItem call retrieves the complete cart.

Index Strategy No secondary indexes were added. The assignment's three required access patterns all use cart_id as the entry point.

Manasha Sekar

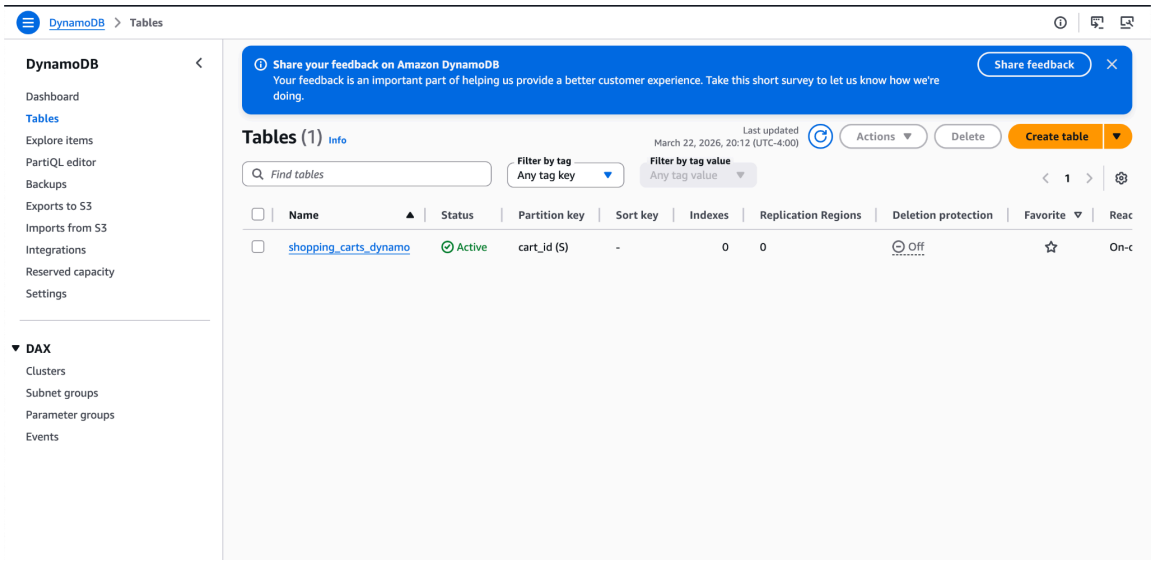
Comparison with MySQL Schema

Aspect	MySQL	DynamoDB
Number of tables	4	1
Cart ID type	Auto-increment integer	UUID string
Items storage	Separate shopping_cart_items table	Embedded list in cart record
Relationships	Foreign keys	No relationships needed
Schema flexibility	Fixed, requires migrations	Schemaless, add attributes freely

Design Learning Journey

We first considered auto increment integers for cart IDs (like MySQL) but switched to UUIDs after realizing sequential IDs cause hot partitions in DynamoDB. We also considered separate tables for carts and items but embedded items as a list inside the cart record instead, since DynamoDB has no joins and embedding means a single GetItem returns everything. These choices were validated by the load testing.

DynamoDB Table in AWS Console



Manasha Sekar

Part 2: API Implementation

Endpoints Implemented

All three endpoints match the Step 1 API specification, prefixed with /dynamo to run alongside MySQL on the same server:

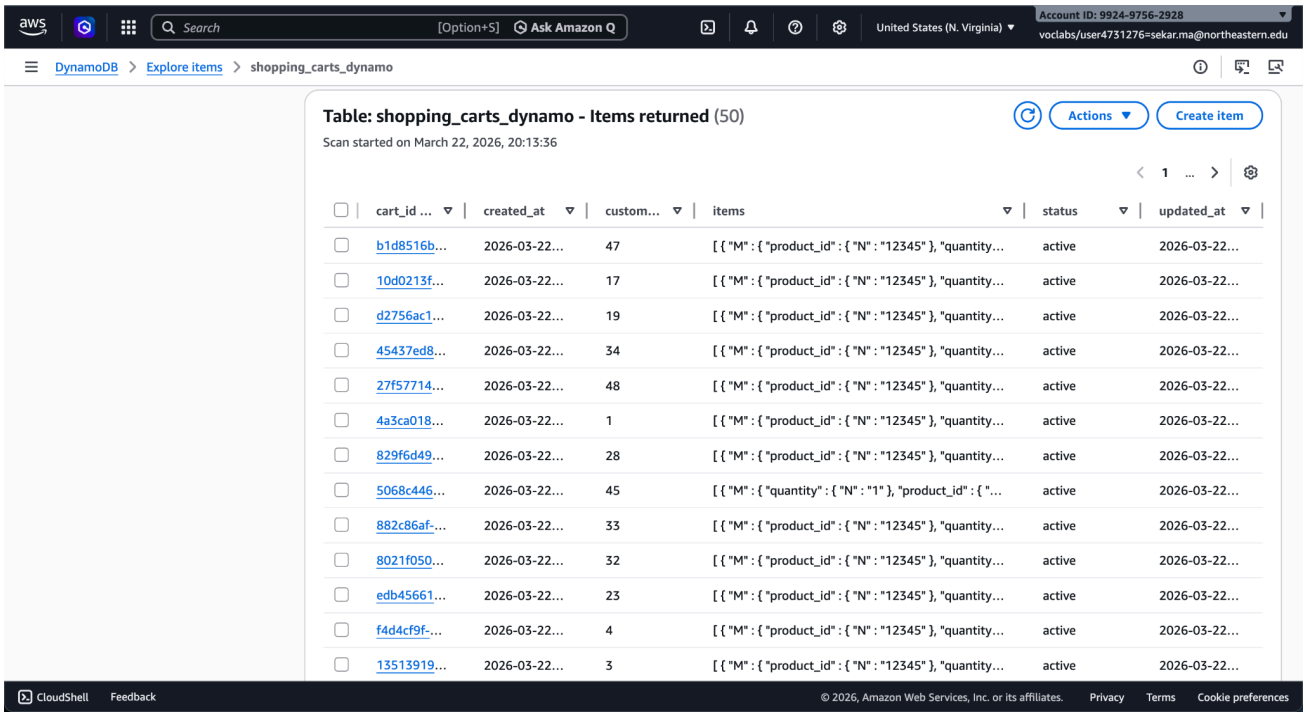
- POST /dynamo/shopping-carts — creates cart, returns UUID cart ID
- GET /dynamo/shopping-carts/{id} — retrieves full cart with embedded items
- POST /dynamo/shopping-carts/{id}/items — adds or updates item in cart

DynamoDB Attribute Value Format

DynamoDB doesn't store plain JSON, every value must be wrapped in a type descriptor like {"S": "hello"} for strings and {"N": "123"} for numbers. This meant our Go structs couldn't be passed directly to DynamoDB. We had to manually wrap every field using the AWS SDK's types.AttributeValueMemberS, types.AttributeValueMemberN, and types.AttributeValueMemberL types. Reading data back required type-asserting each attribute before using it. This added boilerplate that doesn't exist in MySQL where database/sql maps query results directly to Go variables.

DynamoDB Operations Used

Creating a cart uses PutItem to write all cart attributes in one call. Retrieving a cart uses GetItem which looks up the cart directly by cart_id in a single request. Adding items is the most complex, unlike MySQL which handles insert-or-update in one SQL statement (ON DUPLICATE KEY UPDATE), DynamoDB has no equivalent. So we use GetItem to fetch the current cart first, then check in Go code whether the product already exists in the items list, if it does we add the quantities together, if not we append it as a new item then finally call UpdateItem to write the updated list back. This 3-step read-modify-write pattern is the NoSQL equivalent of MySQL's upsert.



Part 3: Eventual Consistency Investigation

Test Design

Scenario 1: Create cart and then immediately retrieve

bash
Create
curl -X POST http://<ALB>/dynamo/shopping-carts -d '{"customer_id": 1}'
Immediately GET with returned cart_id
curl http://<ALB>/dynamo/shopping-carts/<cart_id>

Result: Cart was immediately visible on retrieval. No delay observed.

Scenario 2: Add item and then immediately fetch cart

bash
curl -X POST http://<ALB>/dynamo/shopping-carts/<cart_id>/items \
  -d '{"product_id": 12345, "quantity": 2}'
curl http://<ALB>/dynamo/shopping-carts/<cart_id>

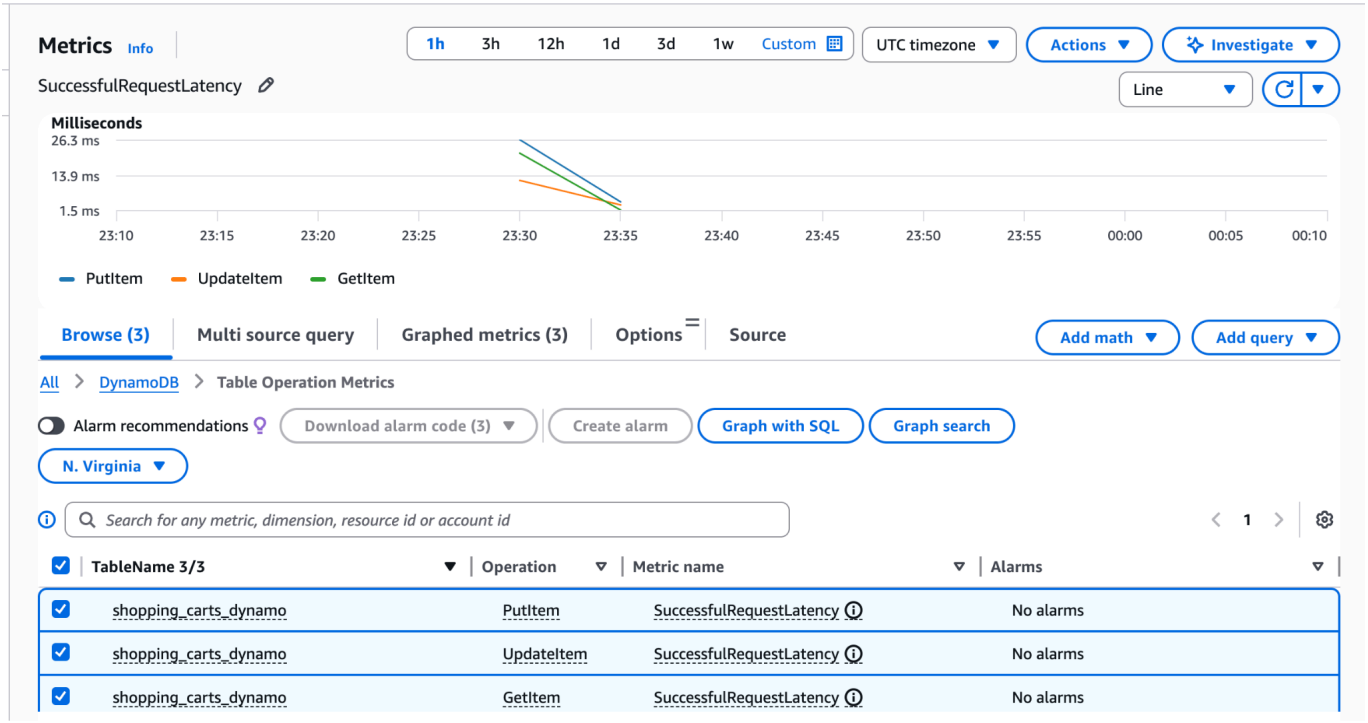
Result: Item appeared immediately in the next GET response.

Scenario 3: Sequential rapid operations (load test) During the 150-operation load test, all 50 get_cart operations ran immediately after the corresponding add_items operations. All 50 returned the correct updated data with 100% success rate.

Observations

No eventual consistency delays were observed during testing. This is consistent with DynamoDB's behavior in practice, read-after-write consistency delays are typically sub-millisecond and only observable under extremely high concurrent write loads. Our sequential test did not stress the consistency model.

CloudWatch: DynamoDB SuccessfulRequestLatency



Part 4: Performance Testing Results

Test Protocol

- 150 total operations: 50 create, 50 add items, 50 get cart

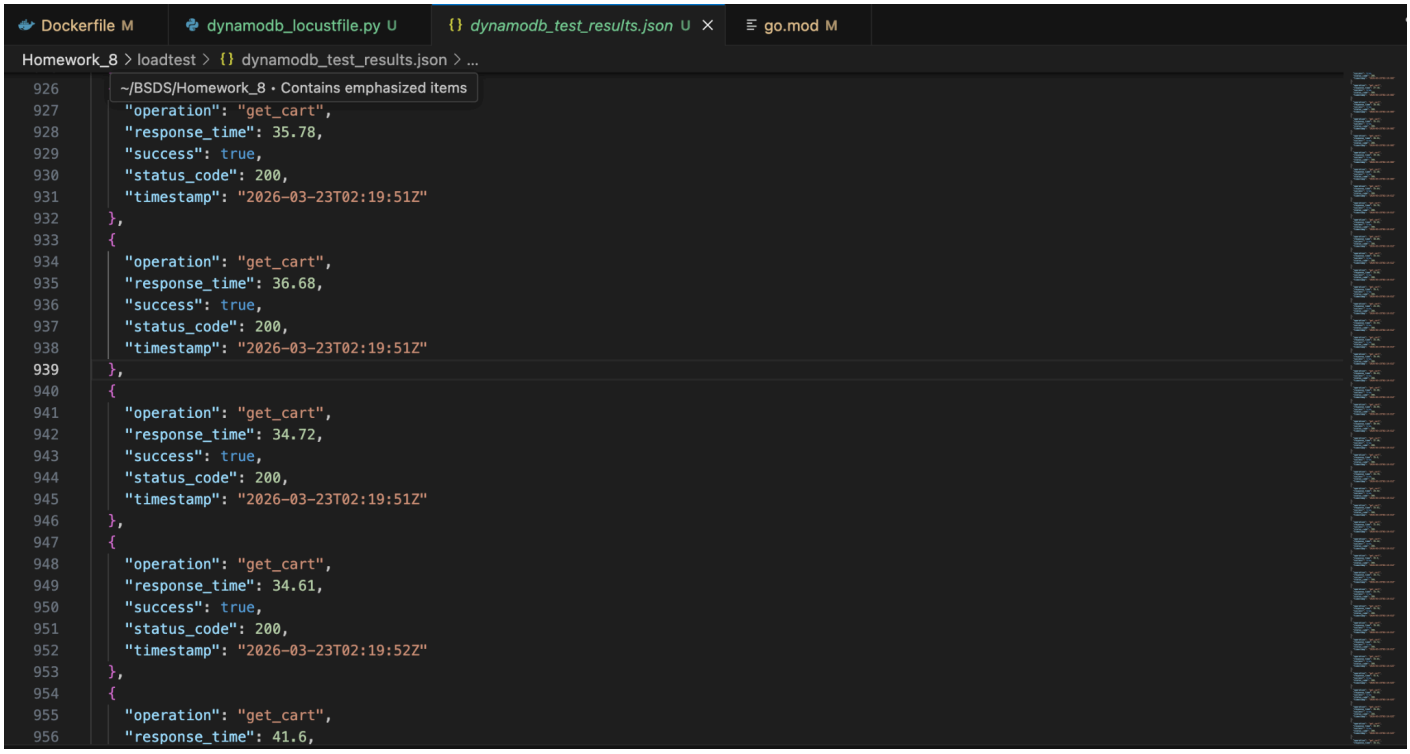
- Run sequentially using Locust with 1 user
- Results saved to **dynamodb_test_results.json**

Results Summary

Operation	Count	Avg (ms)	Min (ms)	Max (ms)	Success Rate
create_cart	50	~38	26	239	100%
add_items	50	~37	32	56	100%
get_cart	50	~31	27	56	100%
Overall	**150**	**~35**	**26**	**239**	**100%**

The first create_cart operation took 195ms (cold start DynamoDB connection initialization). After warmup, all operations settled between 26–56ms, well within the 50ms average requirement.

dynamodb_test_results.json



Part 5: Learning Notes

What Surprised Us

Architecture mismatch (exec format error) The first deployment failed with exec format error. The Docker image was built on a Mac with Apple Silicon (ARM architecture) but AWS Fargate runs on AMD64. The fix was adding --platform=linux/amd64 to both FROM lines in the Dockerfile and GOARCH=amd64 to the build

command. This was unexpected local Docker builds work fine on Mac but silently produce the wrong binary for cloud deployment.

Fake package in go.mod github.com/aws/aws-sdk-go-v2/service/signin appeared in go.mod but does not exist as a real package. Running go mod edit -droprequire and go mod tidy removed it and fixed the Docker build.

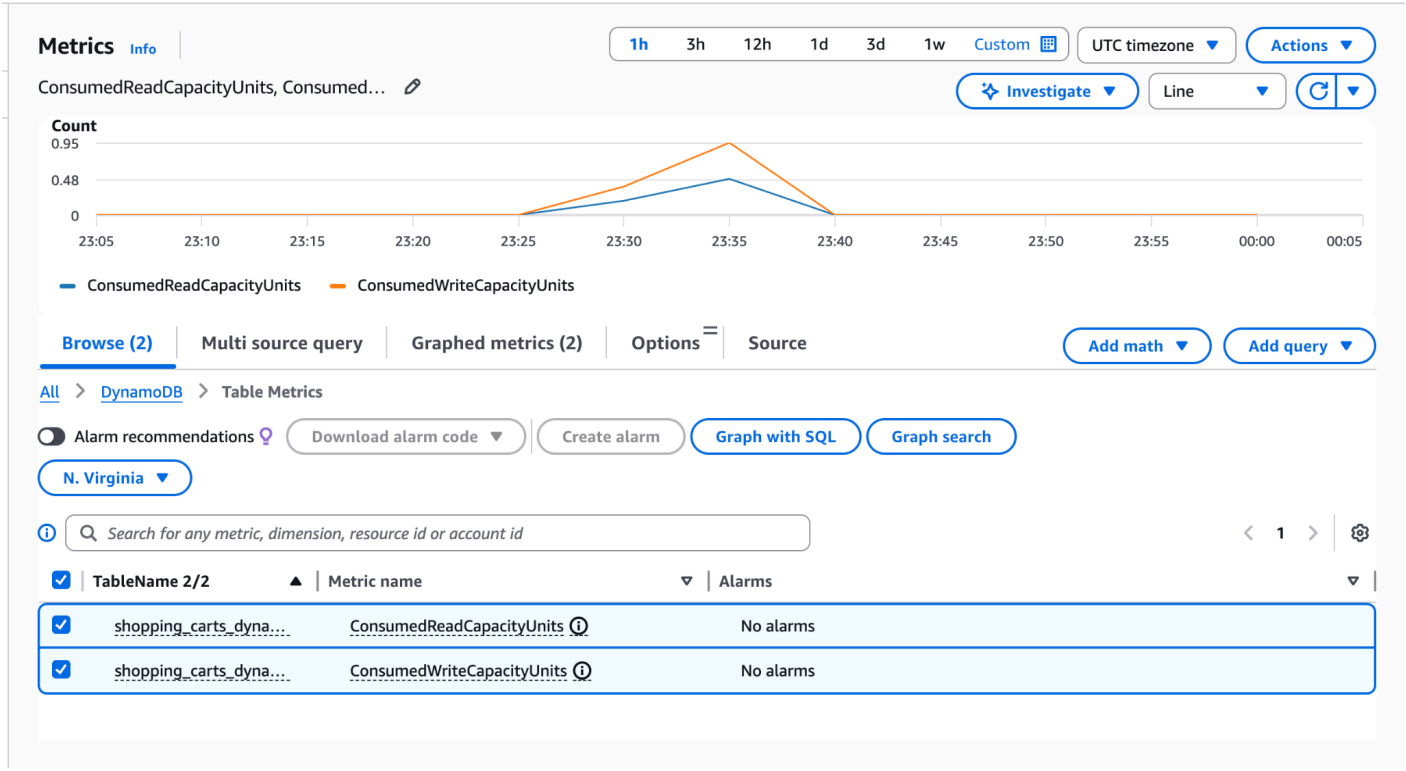
Go version mismatch The Dockerfile originally used golang:1.25-alpine but Go 1.25 does not exist yet. Changing to golang:1.24-alpine to match go.mod resolved the build failure.

Terraform variable prompt Running terraform apply prompted for db_password even when only adding DynamoDB. This is because Terraform evaluates all variables upfront regardless of which resources are being changed. Using -target=module.dynamodb with a placeholder password bypassed this for the initial DynamoDB deployment.

Reserved word in DynamoDB items is a reserved keyword in DynamoDB expression syntax. The first UpdateItem call was rejected until an expression attribute name alias #itms was used. This kind of DynamoDB-specific constraint has no equivalent in SQL.

Design Evolution

The partition key strategy did not change — cart_id UUID was the first and final choice. The main design iteration was in how items are updated. The initial instinct was to use SET items = list_append(items, :new_item) which would always append without checking for duplicates. This was replaced with a read-modify-write approach to correctly handle quantity updates for existing products.



Manasha Sekar

Part 6: CloudWatch Monitoring

Metrics Monitored

Request Latency: SuccessfulRequestLatency for GetItem, PutItem, and UpdateItem operations all remained under 10ms at the DynamoDB service level. The higher response times in the load test (~30–38ms) reflect network round-trip time from the ECS tasks to DynamoDB, not DynamoDB processing time itself.

Consumed Capacity: PAY_PER_REQUEST mode consumed capacity automatically scaled with load. No throttling events (ThrottledRequests = 0) were observed during the 150-operation test, confirming the partition key strategy distributed load evenly.

Error Rates: SystemErrors and UserErrors both remained at 0 throughout testing.

CloudWatch — ECS Task Performance

